

```
"""
```

```
TalentScan AI – High-Level Review Page v10
```

```
Enhancements:
```

1. Filters: name, email, major, degree, years of experience, location, gender
2. Rich Excel export: No, Name, Mobile, Email, Major, Degree, YOE, Current/Last Category, Suitable Dept, Priority, Status, Note, All Scores, Matching Potent AI Rationale, Skills Match, Gaps
3. Table view updates live with filters
4. All known bugs resolved

```
"""
```

```
import csv, html as html_module, io, json, os, time
from datetime import datetime
import openpyxl
from openpyxl.styles import Alignment, Border, Font, PatternFill, Side
from openpyxl.utils import get_column_letter
import requests
import streamlit as st
import streamlit.components.v1 as components
import plotly.graph_objects as go
from reportlab.lib.pagesizes import A4
from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
from reportlab.lib.units import mm
from reportlab.lib import colors
from reportlab.platypus import (
    SimpleDocTemplate, Paragraph, Spacer, Table, TableStyle, HRFlowable
)

st.set_page_config(page_title="TalentScan · High-Level Review", page_icon="📄",
                    layout="wide", initial_sidebar_state="expanded")

API_BASE = os.getenv("API_URL", "http://api:8000")
API      = f"{API_BASE}/api/v1"

# — Helpers —————
def score_color(s):
    return "#42B72A" if s >= 0.65 else ("#F5A623" if s >= 0.45 else "#FA3E3E")

def fit_label(s):
    if s >= 0.65: return ("Strong Fit", "strong")
    if s >= 0.45: return ("Moderate Fit", "moderate")
    return          ("Weak Fit", "weak")

def parse_list(val):
```

```

    if isinstance(val, list): return val
    if not val: return []
    try:
        r = json.loads(val); return r if isinstance(r, list) else [str(r)]
    except: return [str(val)] if val else []

def initials(name):
    parts = (name or "?").split()
    return "".join(p[0] for p in parts[:2]).upper()

def esc(val):
    if val is None or val == "-": return "-"
    return html_module.escape(str(val))

def short_rationale(text, max_chars=160):
    if not text: return "-"
    text = text.strip()
    for sep in [". ", "! ", "? "]:
        idx = text.find(sep)
        if 0 < idx < max_chars: return esc(text[:idx + 1])
    return esc((text[:max_chars] + "...") if len(text) > max_chars else text)

def plain(val):
    return "" if val is None or val == "-" else str(val).strip()

def safe_float(val, default=0.0):
    try: return float(str(val).split()[0])
    except: return default

def priority_label(comp):
    if comp >= 0.75: return "P1 - Urgent"
    if comp >= 0.65: return "P2 - High"
    if comp >= 0.50: return "P3 - Medium"
    if comp >= 0.35: return "P4 - Low"
    return "P5 - Archive"

def category_label(comp):
    if comp >= 0.65: return "Shortlisted"
    if comp >= 0.45: return "Review Needed"
    return "Not Recommended"

def get_parsed(r):
    p = r.get("parsed_data") or {}
    if isinstance(p, str):
        try: p = json.loads(p)
        except: p = {}
    return p

```

```

def suitable_dept(r):
    """
    Returns the best-fit department based ONLY on CV evidence.
    Priority order:
        1. Use no_jd_suggested_depts if present (computed from CV keyword overlap w
        2. Use no_jd_best_cluster if present
        3. Fall back to CV keyword scan against org taxonomy clusters
        4. "General" only if no evidence found
    Never infers or guesses – all results traceable to CV content.
    """
    # 1. Use system-computed dept match (from no_jd_agent org taxonomy scan)
    suggested = r.get("no_jd_suggested_depts") or []
    if isinstance(suggested, list) and suggested:
        top = suggested[0]
        if isinstance(top, dict):
            return top.get("cluster") or top.get("department") or "General"

    # 2. Use best_cluster from score_breakdown
    sb = r.get("score_breakdown") or {}
    if isinstance(sb, str):
        try: sb = json.loads(sb)
        except: sb = {}
    best = sb.get("best_cluster") or r.get("no_jd_best_cluster")
    if best:
        return best

    # 3. Evidence-only keyword scan on actual parsed CV fields (no role-title gue
    parsed = get_parsed(r)
    cv_fields = []
    for field in ["technical_skills", "soft_skills", "domain_expertise", "certificat
        cv_fields.extend(parse_list(parsed.get(field)))
    for exp in (parsed.get("experience") or []):
        cv_fields.append(str(exp.get("role") or ""))
        cv_fields.append(str(exp.get("description") or ""))
    for edu in (parsed.get("education") or []):
        cv_fields.append(str(edu.get("field") or ""))
        cv_fields.append(str(edu.get("degree") or ""))
    cv_blob = " ".join(cv_fields).lower()

    # FIX-2: Corrected org taxonomy keyword clusters
    # Changes: expanded HR cluster (emiratization/emiratisation + MOHRE/NAFIS),
    #           expanded Supply Chain cluster (logistics management degree title),
    #           narrowed Marketing cluster (removed generic 'communications').
    CLUSTERS = [
        ("mechanical engineering", "electrical engineering", "systems engineering"
        ("missiles", "weapons", "defence", "munitions", "radar", "guided systems", "wa

```

```

(["flight test","prototype","telemetry","flight dynamics","airworthiness
(["manufacturing","production","machining","fabrication","cnc","lean","as
(["software","python","java","cloud","azure","cybersecurity","devops","da
(["project management","program management","pmp","agile","scrum","pmo","
(["quality management","iso 9001","as9100","safety management","qhse","au
# FIX-2: 'logistics management' added to match degree title exactly
(["supply chain","logistics management","logistics operations","procureme
(["finance","accounting","ifrs","gaap","tax","treasury","budgeting","fina
# FIX-2: Added UAE-specific HR terms; both 'emiratization' and 'emiratisa
(["human resources","hr management","recruitment","talent acquisition","l
    "emiratisation","emiratization","mohre","nafis","onboarding","employee
# FIX-2: Narrowed to explicit marketing terms – removed generic 'communic
(["marketing campaign","brand management","brand development","social med
(["business development","sales","commercial","contracts","government con
(["facilities management","maintenance","hvac","construction","infrastruc
(["legal","corporate law","contract law","regulatory affairs"], "Legal &
(["uav","drone","unmanned","uas","rpas","counter-uav","autonomous systems
# FIX-2: New cluster for Government Relations / Public Affairs profiles
(["government relations","public affairs","regulatory compliance","stakeh
    "government communication","government policy","institutional relations
]
best_cluster, best_hits = "General", 0
for keywords, cluster in CLUSTERS:
    hits = sum(1 for kw in keywords if kw in cv_blob)
    if hits > best_hits:
        best_hits = hits
        best_cluster = cluster
return best_cluster

# — Excel export —————
EXPORT_COLUMNS = [
    "No","Name","Mobile","Email","Major","Degree",
    "Years of Experience","Current / Last Position","Current Employer",
    "Category","Suitable Department","Priority","Status","Note",
    "AI Score (%)","Relevance Score (%)","Semantic Score (%)","Potential Score (%)
    "Matching Potential – Composite","Matching Potential – Relevance","Matching P
    "Fit Status","AI Rationale","Skills Match","Gaps",
]

def results_to_rows(results):
    rows = []
    for i, r in enumerate(results, 1):
        comp = r.get("composite_score", 0)
        rel = r.get("relevance_score", 0)
        sem = r.get("semantic_similarity", 0)
        pot = r.get("potential_score", 0)
        fit_text, _ = fit_label(comp)

```

```

parsed = get_parsed(r)
edu_list = parsed.get("education") or []
strengths = parse_list(r.get("strengths") or [])
gaps      = parse_list(r.get("gaps") or [])
tech      = parse_list(r.get("technical_skills") or parsed.get("technical_skills"))
degree_raw = plain(r.get("degree") or (edu_list[0].get("degree") if edu_list else ""))
# FIX-3: Append (In Progress) when education status indicates incomplete
degree_status = (edu_list[0].get("status") or "") if edu_list else ""
degree = (degree_raw + " (In Progress)") if degree_status == "in_progress" else degree_raw
major = plain(r.get("major") or (edu_list[0].get("field") if edu_list else ""))
phone = plain(r.get("phone") or parsed.get("phone") or "")
exp_list = parsed.get("experience") or []
# FIX-5: Extended Skills Match fallback chain – never produce blank
skills_match_items = (
    strengths
    or parse_list(r.get("transferable_skills"))
    or tech
    or parse_list(parsed.get("certifications"))
    or parse_list(parsed.get("domain_expertise"))
    or []
)
rows.append({
    "No": str(i),
    "Name": plain(r.get("full_name") or r.get("original_filename") or ""),
    "Mobile": phone,
    "Email": plain(r.get("email")),
    "Major": major,
    "Degree": degree,
    "Years of Experience": plain(r.get("years_of_experience") or parsed.get("years_of_experience") or ""),
    "Current / Last Position": plain(r.get("current_designation") or (exp_list[0].get("current_position") if exp_list else "")),
    "Current Employer": plain(r.get("current_employer") or (exp_list[0].get("current_employer") if exp_list else "")),
    "Category": category_label(comp),
    "Suitable Department": suitable_dept(r),
    "Priority": priority_label(comp),
    "Status": plain(r.get("decision", "")).replace("_", " ").title(),
    "Note": "",
    "AI Score (%)": str(int(comp*100)),
    "Relevance Score (%)": str(int(rel*100)),
    "Semantic Score (%)": str(int(sem*100)),
    "Potential Score (%)": str(int(pot*100)),
    "Matching Potential – Composite": f"{comp:.2f}",
    "Matching Potential – Relevance": f"{rel:.2f}",
    "Matching Potential – Semantic": f"{sem:.2f}",
    "Fit Status": fit_text,
    "AI Rationale": plain(r.get("llm_rationale") or ""),
    "Skills Match": "; ".join(skills_match_items[:10]),
    "Gaps": "; ".join(gaps[:10]),
})

```

```

    })
    return rows

```

```

def build_excel(results, role_name=""):
    wb = openpyxl.Workbook(); ws = wb.active; ws.title = "Candidate Review"
    NAVY="0F2D6B"; BLUE="1877F2"; GREEN_BG="E6F9EE"; GREEN_FG="1A7A3A"
    AMBER_BG="FFF4E5"; AMBER_FG="B35900"; RED_BG="FFECEC"; RED_FG="C62828"
    HEADER_BG="F7F8FC"; SCORE_HDR="EBF3FF"; GRAY="8A9BB0"; BORDER_C="E2E6EA"
    thin = Side(border_style="thin", color=BORDER_C); bdr = Border(left=thin, right=thin, top=thin, bottom=thin)
    last_col = get_column_letter(len(EXPORT_COLUMNS))
    for row_n, (val, fill, fnt) in enumerate([
        ("TalentScan AI – Candidate Review Report",
         PatternFill("solid", fgColor=NAVY), Font(name="Calibri", bold=True, size=16),
         (f"Role: {role_name} | Exported: {datetime.now().strftime('%d %b %Y',
         PatternFill("solid", fgColor=BLUE), Font(name="Calibri", size=10, color="F
    ], 1):
        ws.merge_cells(f"A{row_n}:{last_col}{row_n}")
        c = ws[f"A{row_n}"]; c.value = val; c.fill = fill; c.font = fnt
        c.alignment = Alignment(horizontal="left", vertical="center", indent=1)
    ws.row_dimensions[1].height = 32; ws.row_dimensions[2].height = 22; ws.row_dimensions[3].height = 22; ws.row_dimensions[4].height = 22
    HDR = 4
    SCORE_COLS = {"AI Score (%)", "Relevance Score (%)", "Semantic Score (%)", "Matching Potential – Composite", "Matching Potential – Relevance"}
    POT_COLS = {"Matching Potential – Composite", "Matching Potential – Relevance"}
    for ci, cn in enumerate(EXPORT_COLUMNS, 1):
        cell = ws.cell(row=HDR, column=ci, value=cn)
        cell.font = Font(name="Calibri", bold=True, size=9, color=GRAY)
        cell.fill = PatternFill("solid", fgColor=SCORE_HDR if cn in SCORE_COLS else POT_COLS)
        cell.alignment = Alignment(horizontal="center", vertical="center", wrap_text=True)
        cell.border = bdr
    ws.row_dimensions[HDR].height = 30
    WRAP_COLS = {"AI Rationale", "Skills Match", "Gaps", "Note"}
    col_widths = {"No":5, "Name":22, "Mobile":15, "Email":28, "Major":18, "Degree":16, "Years of Experience":10, "Current / Last Position":26, "Current Category":16, "Suitable Department":18, "Priority":14, "Status":14, "AI Score (%)":10, "Relevance Score (%)":12, "Semantic Score (%)":12, "Matching Potential – Composite":14, "Matching Potential – Relevance":14, "Fit Status":14, "AI Rationale":48, "Skills Match":40, "Gaps":36}
    for ri, row_data in enumerate(results_to_rows(results), HDR+1):
        fit = row_data["Fit Status"]
        row_fill = PatternFill("solid", fgColor=GREEN_BG if fit=="Strong Fit" else AMBER_BG if fit=="Moderate Fit" else RED_BG)
        fit_fc = GREEN_FG if fit=="Strong Fit" else AMBER_FG if fit=="Moderate Fit" else RED_FG
        for ci, cn in enumerate(EXPORT_COLUMNS, 1):
            val = row_data[cn]; cell = ws.cell(row=ri, column=ci, value=val)
            cell.border = bdr; cell.font = Font(name="Calibri", size=10)
            cell.alignment = Alignment(vertical="top", wrap_text=(cn in WRAP_COLS))
            if cn == "No":
                cell.font = Font(name="Calibri", bold=True, size=10); cell.alignment =

```

```

        elif cn in SCORE_COLS:
            sv = int(val) if val else 0
            sc = GREEN_FG if sv>=65 else (AMBER_FG if sv>=45 else RED_FG)
            cell.value = sv/100; cell.number_format="0%"
            cell.font = Font(name="Calibri",bold=True,size=10,color=sc)
            cell.alignment = Alignment(horizontal="center",vertical="top"); c
        elif cn in POT_COLS:
            fv = float(val) if val else 0
            sc = GREEN_FG if fv>=0.65 else (AMBER_FG if fv>=0.45 else RED_FG)
            cell.value = fv; cell.number_format="0.00"
            cell.font = Font(name="Calibri",bold=True,size=10,color=sc)
            cell.alignment = Alignment(horizontal="center",vertical="top"); c
        elif cn == "Fit Status":
            cell.font = Font(name="Calibri",bold=True,size=10,color=fit_fc);
            cell.alignment=Alignment(horizontal="center",vertical="top")
        elif cn in ("AI Rationale","Skills Match","Gaps","Note"):
            cell.fill=PatternFill("solid",fgColor="FAFBFC")

    ws.row_dimensions[ri].height = 52
    for ci, cn in enumerate(EXPORT_COLUMNS, 1):
        ws.column_dimensions[get_column_letter(ci)].width = col_widths.get(cn, 16)
    ws.freeze_panes = f"A{HDR+1}"
    ws.auto_filter.ref = f"A{HDR}:{last_col}{HDR}"
    buf = io.BytesIO(); wb.save(buf); buf.seek(0); return buf.getvalue()

def build_csv(results):
    buf = io.StringIO(); writer = csv.DictWriter(buf, fieldnames=EXPORT_COLUMNS)
    writer.writeheader(); writer.writerows(results_to_rows(results))
    return buf.getvalue().encode("utf-8-sig")

# — CSS —————
st.markdown("""
<style>
@import url('https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700;
html,body,[class*="css"]{font-family:'Inter',-apple-system,sans-serif;font-size:1
.stApp{background:#F7F8FC!important;}
.block-container{padding:0 1.5rem 3rem!important;max-width:1440px;}
header,[data-testid="stHeader"]{background:#FFFFFF!important;border-bottom:1px so
section[data-testid="stSidebar"]{background:#FFFFFF!important;border-right:1px so
.ts-nav{background:linear-gradient(135deg,#0F2D6B 0%,#1877F2 100%);padding:0 1.5r
.ts-nav-logo{font-size:1.4rem;font-weight:800;color:#FFFFFF;letter-spacing:-0.5px
.ts-nav-badge{font-size:0.65rem;font-weight:700;background:rgba(255,255,255,0.2);
.ts-nav-title{font-size:0.9rem;font-weight:600;color:rgba(255,255,255,0.9);}
.ts-nav-role{font-size:0.78rem;color:rgba(255,255,255,0.55);}
.exec-metric{background:#FFFFFF;border-radius:10px;border:1px solid #E2E6EA;paddi
.exec-metric::before{content:'';position:absolute;top:0;left:0;right:0;height:3px
.exec-metric-val{font-size:2rem;font-weight:800;color:#050505;line-height:1;}
.exec-metric-lbl{font-size:0.68rem;font-weight:700;text-transform:uppercase;lette

```

```
.exec-metric-sub{font-size:0.72rem;color:#BCC0C4;margin-top:2px;}
.filter-panel{background:#FFFFFF;border-radius:10px;border:1px solid #E2E6EA;padd
.filter-title{font-size:0.72rem;font-weight:700;text-transform:uppercase;letter-s
.divider{height:1px;background:#E2E6EA;margin:1.2rem 0;}
.sec-label{font-size:0.68rem;font-weight:700;text-transform:uppercase;letter-spac
.empty-state{text-align:center;padding:4rem 2rem;background:#FFFFFF;border-radius
label{color:#65676B!important;font-size:0.78rem!important;}
.stSelectbox>div>div{background:#FFFFFF!important;border-color:#E2E6EA!important;
.stTextInput>div>div{background:#FFFFFF!important;border-color:#E2E6EA!important;
.stTextInput input{color:#050505!important;}
.stButton>button{background:#1877F2!important;color:#FFFFFF!important;border:none
.result-meta{display:flex;align-items:center;justify-content:space-between;backgr
</style>
""" , unsafe_allow_html=True)
```

```
# — Sidebar —————
with st.sidebar:
    st.markdown("""
    <div style="padding:0.5rem 0 1.2rem;">
        <div style="font-size:1.4rem;font-weight:900;color:#1877F2;letter-spacing:-
        <div style="font-size:0.7rem;color:#8A9BB0;margin-top:2px;font-weight:600;t
    </div>
    <div style="height:1px;background:#E2E6EA;margin-bottom:1rem;"></div>
    <div style="font-size:0.68rem;font-weight:700;color:#8A9BB0;text-transform:up
    """, unsafe_allow_html=True)
    st.page_link("app.py", label="🏠 Dashboard")
    st.page_link("pages/01_Post_a_Job.py", label="📄 Post a Job")
    st.page_link("pages/02_Upload_CVs.py", label="📁 Upload CVs")
    st.page_link("pages/03_High_Level_Review.py", label="📊 High-Level Review")
    st.page_link("pages/04_Detailed_Insights.py", label="🔍 Detailed Insights")
    st.page_link("pages/05_Candidate_History.py", label="📂 Candidate History")
    st.markdown('<div style="height:1px;background:#E2E6EA;margin:1rem 0;"></div>')
    st.markdown('<div class="sec-label">Active Role</div>', unsafe_allow_html=Tru
    try:
        jobs_resp = requests.get(f"{API}/jobs/", timeout=5)
        jobs = jobs_resp.json() if jobs_resp.status_code == 200 else []
        job_options = {j["title"]: j["id"] for j in jobs}
    except Exception:
        job_options = {}
    if not job_options:
        st.warning("No active roles found. Post a job first."); st.stop()
    selected_label = st.selectbox("Role", list(job_options.keys()), label_visibi
    selected_job_id = job_options[selected_label]
    st.markdown('<div style="height:1px;background:#E2E6EA;margin:1rem 0;"></div>')
    st.markdown('<div class="sec-label">Quick Score Filter</div>', unsafe_allow_h
    min_score_pct = st.slider("Minimum AI Score", 0, 100, 0, step=5, format="%d%
    st.markdown('<div style="height:1px;background:#E2E6EA;margin:1rem 0;"></div>')
```

```

st.markdown('<div class="sec-label">System Status</div>', unsafe_allow_html=True)
try:
    h = requests.get(f"{API_BASE}/health", timeout=4).json()
    api_ok=True; db_ok=h.get("database",False); ai_ok=h.get("ollama",False)
except:
    api_ok=db_ok=ai_ok=False
for lbl, ok in [("API Gateway",api_ok), ("PostgreSQL",db_ok), ("Ollama LLM",ai_
    dot = "#42B72A" if ok else "#FA3E3E"
    st.markdown(f'<div style="display:flex;align-items:center;gap:7px;font-si

# — Fetch —————
try:
    resp = requests.get(f"{API}/screenings/results/{selected_job_id}", params={"m
    all_results = resp.json() if resp.status_code == 200 else []
    if not isinstance(all_results, list): all_results = []
except Exception as e:
    st.error(f"Cannot reach API: {e}"); st.stop()

# — Nav —————
st.markdown(f"""
<div class="ts-nav">
    <div style="display:flex;align-items:center;">
        <div class="ts-nav-logo">TalentScan</div>
        <span class="ts-nav-badge">Executive View</span>
    </div>
    <div class="ts-nav-title">High-Level Review Dashboard</div>
    <div class="ts-nav-role">{esc(selected_label)}</div>
</div>""", unsafe_allow_html=True)

# — Metrics —————
total_cvs = len(all_results)
avg_score = (sum(r.get("composite_score",0) for r in all_results)/total_cvs) if
strong_fit = sum(1 for r in all_results if r.get("composite_score",0)>=0.65)
mod_fit = sum(1 for r in all_results if 0.45<=r.get("composite_score",0)<0.65)
weak_fit = sum(1 for r in all_results if r.get("composite_score",0)<0.45)
c1,c2,c3,c4,c5 = st.columns(5)
for col,val,lbl,sub,accent in [
    (c1,total_cvs,"Total CVs","Screened for role","#1877F2"),
    (c2,strong_fit,"Strong Fit","Score ≥ 65%","#42B72A"),
    (c3,mod_fit,"Moderate Fit","Score 45-65%","#F5A623"),
    (c4,weak_fit,"Weak Fit","Score < 45%","#FA3E3E"),
    (c5,f"{avg_score:.0%}" if avg_score else "-", "Avg AI Score", "Pool average", "#
]:
    with col:
        st.markdown(f"""<div class="exec-metric" style="--accent:{accent};">
            <div class="exec-metric-val">{val}</div>
            <div class="exec-metric-lbl">{lbl}</div>

```

```

        <div class="exec-metric-sub">{sub}</div>
    </div>""", unsafe_allow_html=True)

st.markdown('<div class="divider"></div>', unsafe_allow_html=True)

# — No-JD banner (if any results are in no-JD mode) —————
_no_jd_count = sum(1 for r in all_results
                    if (r.get("score_breakdown") or {}).get("no_jd_mode") or
                        (r.get("ai_summary") or "").startswith("[NO JD]"))
if _no_jd_count:
    no_jd_html = (
        f'<div style="background:#FFF8E6;border:1px solid #FFD975;border-radius:8'
        f'margin-bottom:1rem;font-size:0.8rem;color:#7A5700;display:flex;align-it'
        f'<span style="font-size:1.2rem;">⚠️</span>'
        f'<div><strong>No-JD Generic Screening Mode:</strong> {_no_jd_count} of {'
        f'without a Job Description. Their scores are generic (max 72%) based on'
        f'not against a specific role. Scores and fit labels should be interprete'
        f'Department suggestions are shown in the Suitable Department column.</di'
    )
    st.markdown(no_jd_html, unsafe_allow_html=True)

# — Filter panel —————
def unique_vals(field):
    s=set()
    for r in all_results:
        v=r.get(field)
        if v and str(v).strip() not in ("","-","None"): s.add(str(v).strip())
    return sorted(s)

st.markdown('<div class="filter-panel">', unsafe_allow_html=True)
st.markdown('<div class="filter-title">🔍 Candidate Filters – table updates live

fa,fb,fc,fd = st.columns(4)
with fa: f_name = st.text_input("Name", placeholder="Filter by name...", key="f_
with fb: f_email = st.text_input("Email", placeholder="Filter by email...", key="f_
with fc:
    major_opts = ["All"] + unique_vals("major")
    f_major = st.selectbox("Major / Field of Study", major_opts, key="f_major")
with fd:
    degree_opts = ["All"] + unique_vals("degree")
    f_degree = st.selectbox("Degree", degree_opts, key="f_degree")

fe,ff,fg,fh = st.columns(4)
with fe: exp_range = st.slider("Years of Experience", 0, 30, (0,30), step=1, key=
with ff:
    loc_opts = ["All"] + list(dict.fromkeys(unique_vals("location")+unique_vals("
    f_location = st.selectbox("Location", loc_opts, key="f_location")

```

```

with fg:
    gen_opts = ["All"] + unique_vals("gender")
    f_gender = st.selectbox("Gender", gen_opts, key="f_gender")
with fh:
    f_fit = st.selectbox("Fit Category", ["All", "Strong Fit", "Moderate Fit", "Weak

st.markdown('</div>', unsafe_allow_html=True)

# — View mode —————
col_mode, _ = st.columns([3, 7])
with col_mode:
    view_mode = st.radio("View Mode", [ All Profiles",  Top 5 CVs"], horiz

if not all_results:
    st.markdown('<div class="empty-state"><div style="font-size:2.5rem;">

```

```

active_filters = sum([
    1 if f_name else 0, 1 if f_email else 0,
    1 if f_major!="All" else 0, 1 if f_degree!="All" else 0,
    1 if f_location!="All" else 0, 1 if f_gender!="All" else 0,
    1 if (exp_range[0],exp_range[1])!=(0,30) else 0,
    1 if f_fit!="All" else 0, 1 if min_score_pct>0 else 0,
])

if "Top 5" in view_mode:
    st.markdown('<div style="display:inline-flex;align-items:center;gap:8px;backg

st.markdown(f'""<div class="result-meta">
    <span>Showing <strong>{len(display_results)}</strong> candidate{"s" if len(display_results)>1 else ""}
    {f"(filtered from {total_cvs})" if active_filters else ""}
</span>
    <span style="font-size:0.75rem;color:#8A9BB0;">
        {"🔵 "+str(active_filters)+" filter(s) active" if active_filters else "No filters"}
    </span>
</div>""", unsafe_allow_html=True)

if not display_results:
    st.info("No candidates match the current filters. Adjust the filters above.")

# — Export —————
st.markdown('<div class="sec-label" style="margin-bottom:10px;">Export</div>', unsafe_allow_html=True)
_ts = datetime.now().strftime("%Y%m%d_%H%M")
_sr = "".join(c if c.isalnum() or c in "-_" else "_" for c in selected_label)[:40]
e1, e2, e3, e4 = st.columns([2, 2, 2, 4])
with e1:
    st.download_button(
        "📄 Download CSV",
        data=build_csv(display_results),
        file_name=f"TalentScan_{_sr}_{_ts}.csv",
        mime="text/csv",
        use_container_width=True,
    )
with e2:
    st.download_button(
        "📄 Download Excel",
        data=build_excel(display_results, role_name=selected_label),
        file_name=f"TalentScan_{_sr}_{_ts}.xlsx",
        mime="application/vnd.openxmlformats-officedocument.spreadsheetml.sheet",
        use_container_width=True,
    )
with e3:
    # PDF always exports ALL filtered candidates (sorted_r), not just the current

```

```

st.download_button(
    "📄 Download PDF Report",
    data=build_pdf(sorted_r, role_name=selected_label),
    file_name=f"TalentScan_{_sr}_{_ts}.pdf",
    mime="application/pdf",
    use_container_width=True,
)
with e4:
    st.markdown(
        f'<div style="font-size:0.72rem;color:#8A9BB0;padding-top:10px;">'
        f'CSV/Excel: <strong style="color:#1A1A2E;">{len(display_results)}</strong>'
        f'&nbsp;&nbsp;  PDF: <strong style="color:#1A1A2E;">{len(sorted_r)}</strong>'
        f'for <strong style="color:#1A1A2E;">{esc(selected_label)}</strong></div>'
        unsafe_allow_html=True,
    )

st.markdown('<div class="divider"></div>', unsafe_allow_html=True)

# — Table —————
def build_table_html(results):
    rows_html = ""
    for rank, r in enumerate(results, 1):
        name = r.get("full_name") or r.get("original_filename") or "Unknown"
        email = r.get("email") or "-"
        phone = r.get("phone") or "-"
        loc = r.get("location") or r.get("city") or "-"
        gender = r.get("gender") or "-"
        exp_yrs = r.get("years_of_experience") or "-"
        emp = r.get("current_employer") or "-"
        des = r.get("current_designation") or "-"
        parsed = get_parsed(r)
        edu = parsed.get("education") or []
        degree = r.get("degree") or (edu[0].get("degree") if edu else "") or "-"
        major = r.get("major") or (edu[0].get("field") if edu else "") or "-"
        comp = r.get("composite_score",0); rel=r.get("relevance_score",0); sem
        rat = r.get("llm_rationale") or ""
        fit_text, fit_cls = fit_label(comp); score_pct=int(comp*100); score_col=s
        ini = esc(initials(name)); rank_cls = f"rank-{rank}" if rank<=3 else
        dept = suitable_dept(r); priority = priority_label(comp)
        fit_dot = "#1A7A3A" if fit_cls=="strong" else ("B35900" if fit_cls=="mod
        exp_sfx = ' <span style="font-size:0.7rem;color:#8A9BB0;"> yrs</span>' if
        rows_html += f"""
<tr>
<td><div class="rank-badge {rank_cls}">#{rank}</div></td>
<td>
<div class="cand-cell">
<div class="mini-av">{ini}</div>

```

```

        <div>
            <div class="cn">{esc(name)}</div>
            <div class="cm">{esc(email)}</div>
            <div class="cm" style="color:#BCC0C4;">{esc(phone)}</div>
        </div>
    </td>
    <td><span class="pill">📍 {esc(loc)}</span></td>
    <td><span style="font-size:0.8rem;color:#444;">{esc(str(gender))}</span>
    <td><span style="font-weight:700;font-size:0.85rem;">{esc(str(exp_yrs))}
    <td>
        <div style="font-weight:600;font-size:0.78rem;">{esc(str(degree))}</div>
        <div style="font-size:0.7rem;color:#8A9BB0;">{esc(str(major))}</div>
    </td>
    <td>
        <div style="font-weight:600;font-size:0.8rem;">{esc(emp)}</div>
        <div style="font-size:0.72rem;color:#8A9BB0;">{esc(des)}</div>
    </td>
    <td><div style="font-size:0.72rem;font-weight:700;color:#1877F2;">{esc(
    <td>
        <span class="sp" style="background:{score_col}18;color:{score_col};bo
        <div style="font-size:0.65rem;color:#8A9BB0;margin-top:2px;">R:{int(r
    </td>
    <td>
        <span class="fit fit-{fit_cls}"><span style="width:6px;height:6px;bor
        <div style="font-size:0.65rem;color:#8A9BB0;margin-top:2px;">{esc(pri
    </td>
    <td><div class="rat">{short_rationale(rat)}</div></td>
</tr>""""
h = 44 + len(results)*64 + 32
return f""""<!DOCTYPE html><html><head><meta charset="utf-8">
<link href="https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700;8
<style>
*{{box-sizing:border-box;margin:0;padding:0;}}
body{{font-family:'Inter',-apple-system,sans-serif;font-size:14px;background:tran
.exec-table{{width:100%;border-collapse:separate;border-spacing:0;background:#FFF
.exec-table thead tr{{background:#F7F8FC;}}
.exec-table th{{font-size:0.63rem;font-weight:700;text-transform:uppercase;letter
.exec-table td{{padding:12px 12px;border-bottom:1px solid #F0F2F5;vertical-align:
.exec-table tbody tr:hover{{background:#F7F9FF;}}
.exec-table tbody tr:last-child td{{border-bottom:none;}}
.rank-badge{{width:28px;height:28px;border-radius:50%;background:#F0F2F5;color:#6
.rank-1{{background:#FFF3CD;color:#856404;}}.rank-2{{background:#E9ECEF;color:#49
.mini-av{{width:34px;height:34px;border-radius:50%;background:#1877F2;color:white
.cand-cell{{display:flex;align-items:center;gap:10px;}}
.cn{{font-weight:600;color:#050505;font-size:0.85rem;}}.cm{{font-size:0.7rem;colo
.fit{{display:inline-flex;align-items:center;gap:5px;padding:4px 10px;border-radi

```

```

.fit-strong{{background:#E6F9EE;color:#1A7A3A;border:1px solid #B5E8C9;}}
.fit-moderate{{background:#FFF4E5;color:#B35900;border:1px solid #FFCC80;}}
.fit-weak{{background:#FFECEC;color:#C62828;border:1px solid #FFAAA0;}}
.sp{{display:inline-flex;align-items:center;justify-content:center;min-width:46px
.pill{{background:#F0F2F5;color:#444;padding:3px 9px;border-radius:12px;font-size
.rat{{font-size:0.75rem;color:#444;line-height:1.5;max-width:240px;}}
</style></head><body>
<div style="overflow-x:auto;">
  <table class="exec-table">
    <thead><tr>
      <th>#</th><th>Candidate</th><th>Location</th><th>Gender</th>
      <th>Exp</th><th>Degree / Major</th><th>Current Role / Employer</th>
      <th>Dept</th><th>AI Score</th><th>Fit / Priority</th><th>AI Rationale</th>
    </tr></thead>
    <tbody>{rows_html}</tbody>
  </table>
</div></body></html>""", h

```

```

st.markdown('<div class="sec-label" style="margin-bottom:14px;">Candidate Summary
table_html, table_height = build_table_html(display_results)
components.html(table_html, height=table_height, scrolling=False)

```

```

st.markdown("""
<div style="margin-top:1rem;padding:12px 16px;background:#F0F2F5;border-radius:8p
  <span><img alt="info icon" data-bbox="205 508 225 523"/></span>
  <span>Executive view for fast decisions. For deep analytics use the <strong sty
</div>
""", unsafe_allow_html=True)

```

```

def build_pdf(results, role_name=""):
    """
    Generates a single PDF report containing ALL candidates in a table,
    matching exactly the columns shown on the High-Level Review page.
    """
    buffer = io.BytesIO()
    doc = SimpleDocTemplate(
        buffer, pagesize=A4,
        leftMargin=10*mm, rightMargin=10*mm,
        topMargin=12*mm, bottomMargin=12*mm,
    )
    styles = getSampleStyleSheet()

    NAVY = colors.HexColor("#0F2D6B")
    BLUE = colors.HexColor("#1877F2")
    GREEN = colors.HexColor("#1A7A3A")
    AMBER = colors.HexColor("#B35900")
    RED = colors.HexColor("#C62828")

```

```

LGRAY = colors.HexColor("#F7F8FC")
MGRAY = colors.HexColor("#E2E6EA")
GRAY  = colors.HexColor("#8A9BB0")

small_style = ParagraphStyle("small", parent=styles["Normal"], fontSize=6,
normal_style = ParagraphStyle("normal", parent=styles["Normal"], fontSize=7,
title_style  = ParagraphStyle("title", parent=styles["Title"],  fontSize=14,
                    textColor=colors.white, fontName="Helvetica-Bol
sub_style    = ParagraphStyle("sub",    parent=styles["Normal"], fontSize=8,
                    textColor=colors.white)
hdr_style    = ParagraphStyle("hdr",    parent=styles["Normal"], fontSize=6,
                    fontName="Helvetica-Bold", textColor=GRAY, lead
footer_style = ParagraphStyle("footer", parent=styles["Normal"], fontSize=6,
                    textColor=GRAY, alignment=1)

elements = []

# — Header banner —————
header_data = [[
    Paragraph("TalentScan AI – Candidate Review Report", title_style),
    Paragraph(
        f"Role: {role_name} | Exported: {datetime.now().strftime('%d %b %Y',
        sub_style,
    ),
]]
header_table = Table(header_data, colWidths=[120*mm, 61*mm])
header_table.setStyle(TableStyle([
    ("BACKGROUND", (0, 0), (-1, -1), NAVY),
    ("VALIGN", (0, 0), (-1, -1), "MIDDLE"),
    ("TOPPADDING", (0, 0), (-1, -1), 8),
    ("BOTTOPPADDING", (0, 0), (-1, -1), 8),
    ("LEFTPADDING", (0, 0), (-1, -1), 8),
]))
elements.append(header_table)
elements.append(Spacer(1, 6))

# — Summary metric row —————
total    = len(results)
strong   = sum(1 for r in results if r.get("composite_score", 0) >= 0.65)
moderate = sum(1 for r in results if 0.45 <= r.get("composite_score", 0) < 0.
weak     = sum(1 for r in results if r.get("composite_score", 0) < 0.45)
avg      = (sum(r.get("composite_score", 0) for r in results) / total) if tot

metric_data = [[
    Paragraph(f"<b>{total}</b><br><font size=6 color='grey'>Total CVs</font>
    Paragraph(f"<b>{strong}</b><br><font size=6 color='grey'>Strong Fit</fon
    Paragraph(f"<b>{moderate}</b><br><font size=6 color='grey'>Moderate Fit<

```

```

Paragraph(f"<b>{weak}</b><br/><font size=6 color='grey'>Weak Fit</font>",
Paragraph(f"<b>{avg:.0%}</b><br/><font size=6 color='grey'>Avg AI Score</
]]
metric_table = Table(metric_data, colWidths=[36.2*mm]*5)
metric_table.setStyle(TableStyle([
    ("BACKGROUND",      (0, 0), (-1, -1), LGRAY),
    ("BOX",              (0, 0), (-1, -1), 0.5, MGRAY),
    ("INNERGRID",        (0, 0), (-1, -1), 0.5, MGRAY),
    ("ALIGN",            (0, 0), (-1, -1), "CENTER"),
    ("VALIGN",           (0, 0), (-1, -1), "MIDDLE"),
    ("TOPPADDING",       (0, 0), (-1, -1), 6),
    ("BOTTPADDING",      (0, 0), (-1, -1), 6),
]))
elements.append(metric_table)
elements.append(Spacer(1, 8))

# — Candidate table — columns mirror the page table exactly —————
COL_WIDTHS = [7*mm, 32*mm, 18*mm, 10*mm, 8*mm, 22*mm, 26*mm, 20*mm, 12*mm, 16

headers = [
    Paragraph("#",          hdr_style),
    Paragraph("Candidate",   hdr_style),
    Paragraph("Location",    hdr_style),
    Paragraph("Gender",      hdr_style),
    Paragraph("Exp",         hdr_style),
    Paragraph("Degree/Major", hdr_style),
    Paragraph("Role/Employer", hdr_style),
    Paragraph("Dept",        hdr_style),
    Paragraph("AI Score",     hdr_style),
    Paragraph("Fit/Priority", hdr_style),
    Paragraph("AI Rationale", hdr_style),
]

table_data = [headers]
row_bgs     = [] # (row_index, bg_color)

for rank, r in enumerate(results, 1):
    comp     = r.get("composite_score", 0)
    rel      = r.get("relevance_score", 0)
    sem      = r.get("semantic_similarity", 0)
    pot      = r.get("potential_score", 0)
    name     = plain(r.get("full_name") or r.get("original_filename") or "Unkn
    email    = plain(r.get("email") or "-")
    phone    = plain(r.get("phone") or get_parsed(r).get("phone") or "-")
    loc      = plain(r.get("location") or r.get("city") or "-")
    gender   = plain(r.get("gender") or "-")
    exp_yrs  = plain(r.get("years_of_experience") or get_parsed(r).get("total_

```

```

parsed = get_parsed(r)
edu     = parsed.get("education") or []
edu0    = edu[0] if edu and isinstance(edu[0], dict) else {}
degree  = plain(r.get("degree") or edu0.get("degree") or "-")
major   = plain(r.get("major") or edu0.get("field") or "-")
exp_list = parsed.get("experience") or []
employer = plain(r.get("current_employer") or (exp_list[0].get("company")
desig    = plain(r.get("current_designation") or (exp_list[0].get("role")
dept     = suitable_dept(r)
fit_text, fit_cls = fit_label(comp)
priority = priority_label(comp)
rationale = plain(r.get("llm_rationale") or "-")
score_pct = int(comp * 100)

if fit_cls == "strong":
    row_bg     = colors.HexColor("#E6F9EE")
    fit_color  = GREEN
elif fit_cls == "moderate":
    row_bg     = colors.HexColor("#FFF4E5")
    fit_color  = AMBER
else:
    row_bg     = colors.HexColor("#FFECEC")
    fit_color  = RED

sc_color = GREEN if comp >= 0.65 else (AMBER if comp >= 0.45 else RED)

# Build hex strings safely
sc_hex = sc_color.hexval()[2:]
fit_hex = fit_color.hexval()[2:]

row_data = [
    Paragraph(f"<b>#{rank}</b>", normal_style),
    Paragraph(
        f"<b>{name}</b><br/>"
        f"<font size=6 color='grey'>{email}</font><br/>"
        f"<font size=5 color='#BCC0C4'>{phone}</font>",
        normal_style,
    ),
    Paragraph(loc, small_style),
    Paragraph(gender, small_style),
    Paragraph(
        f"<b>{exp_yrs}</b>" + ("<font size=5 color='grey'> yrs</font>" if
        small_style,
    ),
    Paragraph(f"<b>{degree}</b><br/><font size=5 color='grey'>{major}</fo
    Paragraph(f"<b>{desig}</b><br/><font size=5 color='grey'>{employer}</
    Paragraph(dept, small_style),

```

```

        Paragraph(
            f"<b><font color='{sc_hex}'>{score_pct}%</font></b><br/>"
            f"<font size=5 color='grey'>R:{int(rel*100)} S:{int(sem*100)} P:{
            small_style,
        ),
        Paragraph(
            f"<b><font color='{fit_hex}'>{fit_text}</font></b><br/>"
            f"<font size=5 color='grey'>{priority}</font>",
            small_style,
        ),
        Paragraph(rationale[:220] + ("..." if len(rationale) > 220 else ""), sm
    ]
    table_data.append(row_data)
    row_bgs.append((len(table_data) - 1, row_bg))

cand_table = Table(table_data, colWidths=COL_WIDTHS, repeatRows=1)

style_cmds = [
    ("BACKGROUND", (0, 0), (-1, 0), LGRAY),
    ("LINEBELOW", (0, 0), (-1, 0), 0.8, BLUE),
    ("INNERGRID", (0, 0), (-1, -1), 0.3, MGRAY),
    ("BOX", (0, 0), (-1, -1), 0.5, MGRAY),
    ("VALIGN", (0, 0), (-1, -1), "TOP"),
    ("TOPPADDING", (0, 0), (-1, -1), 4),
    ("BOTTOPPADDING", (0, 0), (-1, -1), 4),
    ("LEFTPADDING", (0, 0), (-1, -1), 3),
    ("RIGHTPADDING", (0, 0), (-1, -1), 3),
]

for row_idx, bg in row_bgs:
    style_cmds.append(("BACKGROUND", (0, row_idx), (-1, row_idx), bg))

cand_table.setStyle(TableStyle(style_cmds))
elements.append(cand_table)

elements.append(Spacer(1, 8))
elements.append(HRFlowable(width="100%", thickness=0.5, color=MGRAY))
elements.append(Spacer(1, 4))
elements.append(Paragraph(
    f"Generated by TalentScan AI | {datetime.now().strftime('%d %b %Y, %H:%M:%S')}
    footer_style,
))

doc.build(elements)
buffer.seek(0)
return buffer.getvalue()

```

